



OBJETIVOS

- Implementar en C un sistema embebido previamente especificado, utilizando el Raspberry Pi Pico SDK, Visual Studio Code y el proceso de construcción del proyecto.
- Controlar y coordinar entradas y salidas digitales con atención al comportamiento temporal y a la actualización conjunta de múltiples GPIO.
- Estructurar, documentar, compilar, depurar y verificar software embebido en C mediante módulos con responsabilidades e interfaces bien definidas.
- Comparar experimentalmente Arduino, MicroPython y C con el Pico SDK y formular conclusiones sustentadas sobre sus costos de desarrollo, niveles de abstracción, capacidades y limitaciones.

DESCRIPCIÓN

El sistema que deberá implementarse es Pulso de Memoria, exactamente como se definió en la práctica anterior. Se conservan la interfaz física, las reglas de la partida, los nueve niveles, las tres vidas, el tiempo acumulado de respuesta, el incremento de la frecuencia de presentación, el límite temporal de cada intento, el palpito del quinto LED, el reinicio y las señalizaciones definidas por cada equipo. La implementación en C deberá superar las mismas pruebas de aceptación bajo condiciones comparables.

2.1 Continuidad de la ingeniería

Los requisitos, las especificaciones técnicas, los supuestos, las decisiones de diseño, el análisis de alternativas, la arquitectura del sistema y el plan de pruebas elaborados en la práctica anterior continúan vigentes. No deben redactarse nuevamente ni alterarse para acomodarlos al código en C. El equipo deberá ampliar la trazabilidad existente para incorporar la nueva realización y sus evidencias.

Puede ser necesario revisar una decisión interna de software cuando esta dependa del lenguaje o del entorno. Esa revisión no constituye un cambio funcional: deberá conservar el requisito que originó la decisión, registrar la nueva alternativa, justificar la selección y señalar sus consecuencias. Si el equipo detecta un error real en los documentos anteriores, deberá identificarlo como una corrección y no presentarlo como una consecuencia inevitable del cambio de lenguaje.

2.2 Escritura sobre múltiples GPIO

En las implementaciones anteriores, el uso habitual de `digitalWrite` en Arduino y de objetos `Pin` en MicroPython obliga a actualizar los GPIO mediante operaciones individuales. Cuando varios pines representan un único patrón lógico, las llamadas sucesivas introducen estados intermedios durante un breve intervalo y generan sobrecarga. Esto puede afectar especialmente la escritura de patrones luminosos, las líneas compartidas de los displays y las transiciones asociadas a su multiplexación.

Esta situación no es una imposibilidad absoluta de Arduino ni de MicroPython, sino una consecuencia de sus interfaces de alto nivel más comunes. En C, el Raspberry Pi Pico SDK proporciona operaciones con máscaras que permiten modificar, en una sola operación lógica, el conjunto de GPIO seleccionado, dejando intactos los demás. La nueva implementación deberá aprovechar conscientemente esta capacidad cuando varios pines formen la misma palabra o el mismo patrón de salida.

El equipo deberá prestar especial atención a las secciones del código que actualizan varios GPIO relacionados entre sí. No basta con traducir cada llamada de la versión Arduino o MicroPython a una llamada individual equivalente en C. El

informe deberá identificar las operaciones que requieren actualización conjunta, explicar el criterio adoptado y presentar evidencia de su comportamiento.

2.3 Alcance de la nueva comparación

El análisis comparativo iniciado en la práctica anterior deberá ampliarse con la experiencia de desarrollo en C y con Visual Studio Code. La unidad de comparación debe ser el proceso completo: preparación del entorno, organización del proyecto, edición, compilación, enlace, carga, ejecución, diagnóstico de errores, pruebas y mantenimiento. No es metodológicamente válido atribuir una diferencia al lenguaje cuando puede provenir de una biblioteca, de una decisión de arquitectura, de la experiencia del equipo o de una configuración distinta.

PROCEDIMIENTO

El trabajo deberá seguir la Guía metodológica para el desarrollo de prácticas de sistemas embebidos. En esta ocasión no se repite el proceso completo, porque sus productos ya existen. La tarea consiste en revisarlos, conservar su trazabilidad y extenderlos con la implementación y la evidencia correspondientes a C.

3.1 Preparación y revisión

1. Revisar la especificación y los resultados de la práctica anterior. Confirmar que los requisitos, los criterios de aceptación, el hardware y el comportamiento esperado estén identificados antes de modificar el software.
2. Preparar el entorno de trabajo en Visual Studio Code y verificar la instalación del compilador, de CMake, del Raspberry Pi Pico SDK, de las extensiones requeridas y del mecanismo de carga o de depuración. Registrar las versiones utilizadas.
3. Crear un proyecto reproducible que incluya el código fuente, los encabezados, el archivo CMakeLists.txt y los demás archivos de configuración necesarios. Una copia del repositorio deberá poder compilarse sin depender de archivos locales no entregados.
4. Revisar la arquitectura lógica de la versión anterior y decidir qué elementos pueden conservarse conceptualmente y cuáles requieren una implementación propia en C. Documentar las decisiones nuevas o revisadas sin cambiar los requisitos del producto.

3.2 Desarrollo incremental

5. Implementar y verificar de forma aislada el acceso a pulsadores, LEDs, displays y recursos temporales. Registrar las condiciones de prueba, los resultados e incidencias.
6. Identificar los grupos de GPIO que presentan patrones relacionados entre sí. Diseñar y verificar su actualización conjunta mediante las capacidades del SDK, observando posibles estados transitorios y sus efectos en la multiplexación.
7. Integrar progresivamente la lógica del juego, sin sustituir la especificación anterior por el comportamiento accidental del programa. Mantener separadas las responsabilidades de acceso al hardware, de la lógica del juego y de la coordinación general.
8. Generar la documentación del código con Doxygen. La documentación deberá describir módulos, interfaces, parámetros, valores de retorno, efectos secundarios, unidades, precondiciones y supuestos relevantes.
9. Ejecutar el mismo plan de pruebas de aceptación utilizado con Arduino y MicroPython. Cuando una prueba requiera ajustes instrumentales debido al nuevo entorno, conservar el criterio de aceptación y justificar únicamente el cambio en el procedimiento.

3.3 Análisis comparativo

La comparación anterior entre Arduino y MicroPython deberá completarse con una tercera columna correspondiente a C, al SDK de Raspberry Pi Pico y a Visual Studio Code. Para cada dimensión se presentarán observaciones verificables, no solo apreciaciones. Cuando sea posible, se utilizarán mediciones obtenidas con el mismo hardware, montaje, caso de prueba y condiciones temporales.

Dimensión	Aspectos que deben contrastarse
Preparación del entorno	Instalación, dependencias, configuración, reproducibilidad y tiempo requerido para obtener la primera ejecución.
Estructura del proyecto	Organización de archivos, módulos y bibliotecas, configuración de compilación y gestión de dependencias.
Ciclo de desarrollo	Edición, compilación o interpretación, enlace, carga, reinicio y rapidez de la realimentación.
Diagnóstico y depuración	Mensajes de error, localización de fallos, inspección del estado y herramientas disponibles.
Control de hardware	Nivel de abstracción, acceso a periféricos y capacidad para operar varios GPIO como un conjunto.
Comportamiento temporal	Latencia, variabilidad, estabilidad de la multiplexación y cumplimiento de intervalos.
Uso de recursos	Memoria, tamaño del programa, tiempo de inicio y demás magnitudes que puedan obtenerse de forma comparable.
Calidad y mantenimiento	Claridad, modularidad, cantidad de código propio, documentación, pruebas y facilidad de modificación.
Portabilidad y contexto de uso	Dependencia del hardware y del ecosistema, rapidez en el prototipado y adecuación a distintos tipos de producto.

3.4 Informe técnico

Se entregará un informe que documente la continuidad de las prácticas y permita verificar el proceso seguido. No es necesario copiar íntegramente los requisitos, especificaciones ni análisis anteriores; deberán referenciarse mediante sus identificadores y presentarse únicamente las ampliaciones, revisiones y nuevas evidencias. El informe contendrá:

- Configuración del entorno y de las versiones de las herramientas utilizadas.
- Arquitectura del software en C, distribución de responsabilidades e interfaces entre módulos.
- Decisiones nuevas o revisadas, con alternativas, criterios, justificación y relación con los requisitos existentes.
- Identificación y análisis de las operaciones en múltiples GPIO, incluyendo el efecto buscado y la evidencia obtenida.
- Ampliación de la matriz de trazabilidad con la implementación en C, las pruebas ejecutadas y sus evidencias.
- Resultados de las pruebas de componente, de integración y de aceptación, con las desviaciones y las correcciones aplicadas.
- Análisis comparativo consolidado de Arduino, MicroPython y C con Pico SDK, y conclusiones delimitadas por la evidencia.
- Referencias técnicas y anexos necesarios para reproducir la compilación y las pruebas.

3.5 Archivos de entrega

La entrega se realizará en Moodle antes del lunes 21 de septiembre de 2026 mediante un único archivo comprimido. Este deberá contener el informe en PDF, el proyecto completo de Visual Studio Code y Pico SDK, el código fuente y los encabezados, el archivo CMakeLists.txt y los demás archivos de configuración, la documentación generada con Doxygen y las evidencias que no estén incorporadas en el informe. No se aceptarán únicamente ejecutables, capturas de pantalla del código ni enlaces que requieran permisos adicionales.

EVALUACIÓN

La evaluación considerará tanto la equivalencia funcional del producto como la calidad del proceso de ingeniería. Una traducción literal que solo funcione parcialmente no demuestra dominio de C ni del SDK, y una comparación narrativa no compensa la ausencia de implementación o de evidencia.

Criterio	Peso	Desempeño completo	Desempeño parcial	Insuficiente
Implementación funcional en C	25 %	Reproduce el juego por completo y cumple con las pruebas de aceptación.	Presenta fallas menores sin comprometer el núcleo del sistema.	Faltan funciones esenciales, no ejecuta o no puede verificarse.
Equivalencia con las versiones anteriores	10 %	Conserva el hardware, el comportamiento, las restricciones y los criterios de aceptación.	Existen diferencias menores, justificadas o con evidencia incompleta.	Se modificó la funcionalidad para facilitar la implementación.
GPIO y comportamiento temporal	10 %	Usa de forma consciente las operaciones conjuntas; evita transitorios relevantes y aporta evidencia temporal.	La estrategia es funcional, pero el análisis o la evidencia es limitado.	Replica los accesos individuales sin análisis o introduce fallas visibles de temporización.
Aplicación de la metodología	10 %	Extiende la trazabilidad y las decisiones antes de integrar; conserva la coherencia con los artefactos previos.	El proceso es reconocible, pero hay vínculos o productos incompletos.	El proceso fue reconstruido después de programar o contradice los documentos previos.
Pruebas y evidencia	10 %	Incluye pruebas reproducibles de componente, de integración y de aceptación, con resultados suficientes.	Las pruebas son pertinentes, pero faltan condiciones, mediciones o evidencia.	Se afirma que funciona, pero sin pruebas reproducibles.
Arquitectura y calidad del código	10 %	Módulos, interfaces, tipos y responsabilidades están claros; el código es consistente y mantenible.	La estructura es funcional, con acoplamiento o inconsistencias moderadas.	Código monolítico, difícil de seguir o con dependencias no controladas.
Documentación Doxygen	10 %	La documentación generada es útil, consistente y cubre los módulos e interfaces.	Se genera, pero presenta omisiones o comentarios poco informativos.	No se genera o se limita a repetir el código.
Informe y análisis comparativo	15 %	Integra evidencia, compara críticamente los tres entornos y formula conclusiones defendibles.	El informe es comprensible, pero el análisis es descriptivo o incompleto.	Informe incompleto o conclusiones basadas principalmente en preferencias.

La ausencia de una implementación ejecutable en C impide valorar la equivalencia funcional, el comportamiento de los GPIO y una parte sustancial del análisis comparativo, aunque el informe las describa.